

JEGYZŐKÖNYV

Komputergrafika és képfeldolgozás

Képfeldolgozó alkalmazás

Név: **Orosz Kristóf**

Neptunkód: **EYZWG9**

Dátum: **2026.05.15.**

Sárospatak, 2026

Tartalomjegyzék

Bevezetés	3
Fájlok tartalmának leírása	3
Adatbázis létrehozása	4
Backend fájlok	4
db.js	4
login.js	5
server.js	6
register.js	6
JavaFX fájlok	7
Main.java	7
Bejelentkezési logika kezelése	7
Regisztrációs folyamat megvalósítása	8
Bejelentkezési és regisztrációs metódusok	9
HTTP POST kérés összeállítása JSON formátumban	9
Hibaüzenet megjelenítése Alert segítségével	9
Kép kivágása	10
Pixelalapú képfeldolgozás WritableImage használatával	10
Fényerő módosítása képfeldolgozás során	10
Kép kivágása egérkijelölés alapján	11
Kép elmosása átlagoló szűrővel	11
Kép forgatása tetszőleges szögben	12
Bejelentkezés és felhasználónév átadása	13
Felhasználónév megjelenítése az alkalmazásban	13
Visszavonás funkció megvalósítása képelőzmények segítségével	13
Zöld színcsatorna alkalmazása a képre	14
Sliderek eseménykezelése és képmódosító műveletek	14
Felhasznált irodalom	14

Bevezetés

A projektem célja egy olyan képfeldolgozó alkalmazás elkészítése volt, amely lehetőséget biztosít különböző képszerkesztési műveletek egyszerű és gyors végrehajtására. A fejlesztés során egy JavaFX alapú grafikus felhasználói felületet és egy Node.js alapú backend rendszert készítettem, amelyek együtt biztosítják az alkalmazás teljes működését. A program támogatja a felhasználók regisztrációját és bejelentkezését, így az alkalmazás személyre szabott módon használható. A backend feladata az adatbázissal való kommunikáció és a felhasználói adatok kezelése, míg a frontend a képszerkesztési funkciók megvalósítását biztosítja. A projekt során különböző képfeldolgozási műveleteket valósítottam meg, például képforgatást, kivágást, fényerő módosítást és elmosási szűrő alkalmazását. A rendszer lehetőséget nyújt képek betöltésére és mentésére is. A fejlesztés során HTTP alapú kliens-szerver kommunikációt alkalmaztam JSON formátumban történő adatküldéssel.

Fájlok tartalmának leírása

- db.js	Az adatbázis-kapcsolat kezelésére használtam.
- login.js	A felhasználók bejelentkezésének ellenőrzésére használtam.
- register.js	Új felhasználók regisztrációjának kezelésére használtam.
- server.js	A backend szerver indítására és kezelésére használtam.
- Main.java	Az alkalmazás indítására és a főablak elindítására használtam.
- LoginView.java	A bejelentkező felület megjelenítésére használtam.
- EditorView.java	A képszerkesztő felület és műveletek kezelésére használtam.
- TransformOperations.java	A képek geometriai átalakítására használtam.
- FilterOperations.java	Képszűrők, megvalósítására használtam.
- ImageOperations.java	Képek megnyitására és mentésére használtam.
- CropOperations.java	A képek kivágására használtam.
- ColorOperations.java	A képek színének módosítására használtam.
- AuthService.java	A frontend és backend közötti kommunikációra használtam.
- AlertHelper.java	Hibaüzenetek megjelenítésére használtam.

Adatbázis létrehozása

Az alkalmazás működéséhez egy MariaDB adatbázist hoztam létre „kepszerkeszto” néven. Ebben az adatbázisban egy „users” nevű táblát készítettem, amely a felhasználók adatait tárolja. A tábla három mezőből áll: az „id” mező azonosítja a felhasználókat egyedi módon, a „username” mező a felhasználó nevét tartalmazza, míg a „password” mező a jelszót tárolja. Az „id” mezőt elsődleges kulcsként hoztam létre, és automatikus növekedésre állítottam be. A tábla létrehozását SQL utasítással valósítottam meg, majd tesztadatokat is beszűrtam a működés ellenőrzéséhez. A backend a bejelentkezés és regisztráció során ezt a táblát használja az adatok lekérdezésére és mentésére.

```
CREATE DATABASE kepszerkeszto;

USE kepszerkeszto;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255),
    password VARCHAR(255)
);
```

```
MariaDB [kepszerkeszto]> SELECT * FROM users;
+----+-----+-----+
| id | username | password |
+----+-----+-----+
| 8  | Kristóf | 111      |
| 9  | Kristóf | 11       |
| 10 | Orosz Kristóf | 0       |
+----+-----+-----+
3 rows in set (0.001 sec)
```

1. Részlet az adatbázis létrehozásából.

Backend fájlok

A backend fejlesztése során több külön fájlt készítettem, amelyek együtt biztosítják az alkalmazás működését. Az Express keretrendszer segítségével valósítottam meg a szerver és az útvonalak kezelését. Az adatbázis-kapcsolatot egy külön fájlban hoztam létre, amelyet a többi modul is felhasznál. A bejelentkezés és a regisztráció kezelését külön fájlokban valósítottam meg, ahol az adatokat SQL lekérdezésekkel dolgozom fel.

db.js

Ebben a fájlban hoztam létre az alkalmazás adatbázis-kapcsolatát. A MySQL2 modul segítségével kapcsolódok a helyi adatbázisszerverhez, ahol megadom a szükséges adatokat, például a felhasználónevet, jelszót és az adatbázis nevét. A kapcsolat létrejöttekor visszajelzést adok a konzolban arról, hogy a csatlakozás sikeres volt-e. Hiba esetén kiírom a megfelelő hibaüzenetet, amely segít a hibák felismerésében. A létrehozott kapcsolatot exportálom, hogy a projekt többi részében is fel tudjam használni.

```
const mysql = require("mysql2");

const db = mysql.createConnection({
```

```

    host: "localhost",
    user: "root",
    password: "",
    database: "kepszerkeszto"
  });
  db.connect(err => {
    if (err) {
      console.error("ERROR:", err);
    } else {
      console.log("Az adatbázis elindult.");
    }
  });
  module.exports = db;

```

2. Részlet az db.js fájlból.

register.js

Ebben a fájlban készítettem el a felhasználók regisztrációjának kezelését. A kliens által küldött adatokat POST kérés segítségével fogadom, majd eltárolom az adatbázisban. Egy SQL beszűrő utasítást használok az új felhasználó rögzítésére. A művelet során ellenőrzöm, hogy történt-e hiba, és ennek megfelelően adok visszajelzést. Sikeres esetben jelzem, hogy a regisztráció megtörtént.

```

const express = require("express");
const router = express.Router();
const db = require("../db");

router.post("/", (req, res) => {
  const { username, password } = req.body;

  const sql =
    "SELECT * FROM users WHERE username = ? AND password = ?";
  db.query(sql, [username, password], (error, result) => {
    if (error) {
      console.log(error);
      return res.send("false");
    }

    if (result.length > 0) {
      res.send("true");
    } else {
      res.send("false");
    }
  });
});

module.exports = router;

```

3. Részlet az register.js fájlból.

login.js

Ebben a fájlban valósítottam meg a felhasználók bejelentkezésének kezelését. Egy POST kérésen keresztül fogadom a felhasználónév és jelszó adatokat. Ezeket egy SQL lekérdezéssel ellenőrzöm az adatbázisban, hogy létezik-e ilyen felhasználó. Amennyiben találat érkezik, sikeres bejelentkezést jelzek vissza, ellenkező esetben sikertelen választ adok. Hiba esetén kiírom a hibát, és szintén sikertelen választ küldök vissza.

```
const express = require("express");
const router = express.Router();
const db = require("../db");

router.post("/", (req, res) => {
  const { username, password } = req.body;

  const sql =
    "SELECT * FROM users WHERE username = ? AND password = ?";

  db.query(sql, [username, password], (error, result) => {
    if (error) {
      console.log(error);
      return res.send("false");
    }

    if (result.length > 0) {
      res.send("true");
    } else {
      res.send("false");
    }
  });
});

module.exports = router;
```

4. Részlet az login.js fájlból.

server.js

Ebben a fájlban valósítottam meg a szerver működését és indítását. Az Express keretrendszer segítségével hoztam létre a webszervert, és beállítottam az adatfeldolgozás kezelését. A különböző funkciókat külön fájlokban készítettem el, majd itt kapcsoltam össze őket útvonalak segítségével. Ez a fájl biztosítja a kliens és a backend közötti kommunikáció alapját.

```
const express = require("express");
const cors = require("cors");
const bodyParser = require("body-parser");
const registerRoute = require("../register");
const loginRoute = require("../login");

const app = express();

app.use(cors());
```

```

app.use(bodyParser.json());

app.use("/register", registerRoute);
app.use("/login", loginRoute);

app.listen(3000, () => {
    console.log("Server fut: http://localhost:3000");
});

```

5. *Részlet az server.js fájlból.*

JavaFX fájlok

A JavaFX fájlokban valósítottam meg az alkalmazás grafikus felhasználói felületét és a felhasználói interakciókat. A fő osztály indítja el az alkalmazást, majd a bejelentkező felület jelenik meg, ahol a felhasználó beléphet vagy regisztrálhat. A bejelentkezés és regisztráció során a program HTTP kéréseken keresztül kommunikál a backend szerverrel. A szerkesztő felületen különböző képműveleteket valósítottam meg, például forgatást, tükrözést, színezést, szűrőket és átméretezést. A rendszer fájlkezelést is tartalmaz, amely lehetővé teszi képek betöltését, mentését, valamint hibakezelést felugró ablakok segítségével.

Main.java

Ebben a fájlban valósítottam meg az alkalmazás belépési pontját. A JavaFX Application osztályt használok az alkalmazás indításához. A start metódusban indítom el a bejelentkező felületet. A Stage objektumot továbbadom a következő nézetnek, így az ablak kezelése egységes marad. Ez a fájl biztosítja, hogy az alkalmazás megfelelően elinduljon.

```

package org.kepfeldolgozo;

import javafx.application.Application;
import javafx.stage.Stage;
import org.kepfeldolgozo.LoginView;

public class Main extends Application {

    @Override
    public void start(Stage stage) {
        new LoginView(stage);
    }

    public static void main(String[] args) {
        Launch();
    }
}

```

6. *Részlet a Main.java fájlból.*

Bejelentkezési logika kezelése

Ez a kódrészlet a bejelentkezési folyamat működését valósítja meg. A setOnAction eseménykezelő biztosítja, hogy a gomb megnyomásakor lefusson a megfelelő logika. A program meghívja az AuthService.login metódust, amely ellenőrzi a felhasználónév és jelszó helyességét. Az eredmény egy logikai érték (boolean), amely meghatározza a további működést. Sikeres bejelentkezés esetén egy információs üzenet jelenik meg, majd megnyílik az EditorView. Hibás adatok esetén hibaüzenetet kap a felhasználó.

```

loginbutton.setOnAction(event -> {

    boolean success =
        AuthService.login(
            usernamefield.getText(),
            passwordfield.getText()
        );

    if (success) {
        Alert alert =
            new Alert(Alert.AlertType.INFORMATION);

        alert.setContentText("Sikeres belépés!");
        alert.showAndWait();

        new EditorView(primarystage);
    } else {
        Alert alert =
            new Alert(Alert.AlertType.ERROR);

        alert.setContentText("Hibás adatok!");
        alert.showAndWait();
    }
});

```

7. Részlet a Login.java fájlból.

Regisztrációs folyamat megvalósítása

Ez a kódrészlet a felhasználók regisztrációját kezeli. A registerbutton gombhoz eseménykezelő van rendelve, amely a gomb megnyomásakor aktiválódik. A rendszer az AuthService.register metódust hívja meg, amely új felhasználót próbál létrehozni. A visszatérési érték alapján eldől, hogy a művelet sikeres volt-e. Sikeres esetén egy információs üzenet jelenik meg, amely visszajelzést ad a felhasználónak. Hiba esetén egy figyelmeztető üzenet jelenik meg.

```

registerbutton.setOnAction(event -> {

    boolean success =
        AuthService.register(
            usernamefield.getText(),
            passwordfield.getText()
        );

    if (success) {
        Alert alert =
            new Alert(Alert.AlertType.INFORMATION);

        alert.setContentText("Sikeres regisztráció!");
        alert.showAndWait();
    } else {
        Alert alert =
            new Alert(Alert.AlertType.ERROR);

        alert.setContentText("Hiba történt a regisztráció során!");
    }
});

```



```

        alert.showAndWait();
    }

});

```

8. Részlet a Login.java fájlból.

Bejelentkezési és regisztrációs metódusok

Ez a kódrészlet két publikus metódust tartalmaz, amelyek a bejelentkezés és a regisztráció kezeléséért felelősek. Mindkét metódus paraméterként megkapja a felhasználónevet és a jelszót. A tényleges működést nem itt valósítottam meg, hanem egy közös sendrequest nevű metódusban. A különbség a két metódus között csak a meghívott URL-ben van. A login a belépési végpontra, míg a register a regisztrációs végpontra küld kérést.

```

public static boolean login(String username, String password) {
    return sendrequest("http://localhost:3000/login", username, password);
}

public static boolean register(String username, String password) {
    return sendrequest("http://localhost:3000/register", username,
password);
}

```

9. Részlet a AuthService fájlból.

HTTP POST kérés összeállítása JSON formátumban

Ez a kódrészlet egy HTTP kapcsolat létrehozását és konfigurálását mutatja be. A URL objektum segítségével meghatározom a célcímet, majd megnyitom a kapcsolatot. A HttpURLConnection osztály lehetővé teszi HTTP kérések küldését Java-ban. A kérés típusa POST, ami azt jelenti, hogy adatokat küldök a szervernek. A Content-Type beállítás jelzi, hogy JSON formátumban küldöm az adatokat. A setDoOutput(true) engedélyezi az adatok küldését a kérés törzsében. Ezután összeállítottam a JSON stringet, amely tartalmazza a felhasználónevet és jelszót. Ez a rész kulcsfontosságú a kliens-szerver kommunikációban.

```

URL url = new URL(urlstring);
HttpURLConnection connection = (HttpURLConnection) url.openConnection();

connection.setRequestMethod("POST");
connection.setRequestProperty("Content-Type", "application/json");
connection.setDoOutput(true);

String json =
    "{\"username\":\"" + username +
        "\", \"password\":\"" + password + "\"}";

```

10. Részlet a AuthService fájlból.

Hibaüzenet megjelenítése Alert segítségével

Ez a kódrészlet egy segédfüggvényt valósít meg, amely hibaüzenetek megjelenítésére szolgál. A metódus paraméterként egy szöveges üzenetet kap, amelyet a felhasználó számára jelenít meg. Az Alert osztály JavaFX-ben beépített dialógusablakokat biztosít. Ebben az esetben ERROR típusú ablak jön létre, amely vizuálisan is jelzi a hibát. A fejléct nullára állítottam, így csak a lényegi üzenet jelenik meg. A showAndWait() metódus megvárja, amíg a felhasználó bezárja az ablakot. Ez a megoldás egységes hibakezelést tesz lehetővé az alkalmazásban.

```

public static void showerror(String message) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setHeaderText(null);
    alert.setContentText(message);
}

```

```
        alert.showAndWait();
    }
```

11. Részlet a AlertHelper fájlból.

Kép kivágása

Ez a kódrészlet egy kép kivágására szolgáló metódust valósít meg. A függvény bemenetként megkap egy képet, valamint a kivágás kezdő koordinátáit és méreteit. A startx és starty paraméterek határozzák meg, hogy honnan induljon a kivágás. A width és height pedig a kivágott terület szélességét és magasságát adják meg. A WritableImage osztály segítségével új képet hoztam létre a megadott részletből. A getPixelReader() metódus lehetővé teszi az eredeti kép pixeljeinek elérését.

```
public static Image crop(Image image, int startx, int starty, int width,
int height) {
    return new WritableImage(image.getPixelReader(), startx, starty, width,
height);
}
```

12. Részlet a CropOperation.java fájlból

Pixelalapú képfeldolgozás WritableImage használatával

A kódrészlet bemutatja, hogyan lehet egy kép egy részét pixel szinten feldolgozni JavaFX-ben. A WritableImage osztály lehetővé teszi új, módosítható képek létrehozását. A konstruktor paraméterként megkapja az eredeti kép PixelReader objektumát. Ez biztosítja a hozzáférést a kép pixeljeihez.

```
WritableImage newimage =
    new WritableImage(
        image.getPixelReader(),
        startx,
        starty,
        width,
        height
    );
```

13. Részlet a CropOperation.java fájlból

Fényerő módosítása képfeldolgozás során

Ez a kódrészlet a kép fényerejének módosítását valósítja meg pixel szinten. A metódus bemenetként egy képet és egy fényerő értéket kap, amely meghatározza a módosítás mértékét. Először lekérdezem a kép szélességét és magasságát, majd létrehozok egy új WritableImage objektumot. A PixelReader segítségével beolvasom az eredeti kép pixeleit, míg a PixelWriter segítségével az új képet írom. Két egymásba ágyazott ciklussal végigmegyek az összes pixelen. Minden pixel esetén növelem a vörös, zöld és kék színcsatorna értékét a megadott fényerővel.

```
public static Image adjustbrightness(Image image, double brightnessvalue) {
    int width = (int) image.getWidth();
    int height = (int) image.getHeight();

    WritableImage outputimage = new WritableImage(width, height);
    PixelReader pixelreader = image.getPixelReader();
    PixelWriter pixelwriter = outputimage.getPixelWriter();

    for (int xcoordinate = 0; xcoordinate < width; xcoordinate++) {
        for (int ycoordinate = 0; ycoordinate < height; ycoordinate++) {
            Color color = pixelreader.getColor(xcoordinate, ycoordinate);

            pixelwriter.setColor(xcoordinate, ycoordinate, new Color(
                clamp(color.getRed() + brightnessvalue),
                clamp(color.getGreen() + brightnessvalue),
```

```

        clamp(color.getBlue() + brightnessvalue),
        1
    ));
}

return outputimage;
}

```

14. Részlet a CorolOperation.java fájlból

Kép kivágása egérkijelölés alapján

Ez a kódrészlet az egérrel történő képkivágás működését valósítja meg. A `setOnMousePressed` eseménynél eltárolom a kezdő koordinátákat, ahol a felhasználó rákattint a képre. A `setOnMouseReleased` eseménynél meghatározom a kijelölés végpontját. Ezután kiszámolom a kép tényleges méretét és a megjelenített méretet, mert ezek eltérhetnek egymástól. A koordinátákat arányosan átszámolom a valódi képméretre, így pontos kivágást kapok. A `Math.min` és `Math.abs` függvények segítségével biztosítom a helyes irányt és méretet. A művelet előtt elmentem az előző állapotot, hogy vissza lehessen vonni.

```

imageView.setOnMousePressed(event -> {
    startx = event.getX();
    starty = event.getY();
});

imageView.setOnMouseReleased(event -> {

    double endx = event.getX();
    double endy = event.getY();

    int imagewidth = (int) currentimage.getWidth();
    int imageheight = (int) currentimage.getHeight();

    double displayedwidth = imageView.getBoundsInLocal().getWidth();
    double displayedheight = imageView.getBoundsInLocal().getHeight();

    int cropx = (int) (Math.min(startx, endx) * imagewidth / displayedwidth);
    int cropy = (int) (Math.min(starty, endy) * imageheight / displayedheight);
    int cropwidth = (int) (Math.abs(startx - endx) * imagewidth /
        displayedwidth);
    int cropheight = (int) (Math.abs(starty - endy) * imageheight /
        displayedheight);

    savehistory();
    currentimage = CropOperations.crop(currentimage, cropx, cropy, cropwidth,
        cropheight);
    imageView.setImage(currentimage);
});

```

15. Részlet a EditorView.java

Kép elmosása átlagoló szűrővel

Ez a kódrészlet a kép elmosását valósítja meg egy átlagoló szűrő segítségével. A külső két ciklussal végigmegyünk a kép minden egyes pixelén. Minden pixel esetén megvizsgálom a környezetében lévő pixeleket egy adott sugarú tartományban. A belső ciklusok végigiterálnak ezen a szomszédos területen. Az egyes pixelek vörös, zöld és kék színértékeit összeadom, majd megszámlolom, hány pixel vett részt a számításban.

```

for (int xcoordinate = 0; xcoordinate < width; xcoordinate++) {
    for (int ycoordinate = 0; ycoordinate < height;
        ycoordinate++) {

        double redsum = 0;
        double greensum = 0;
        double bluesum = 0;
        int pixelsum = 0;

        for (int horizontaloffset = -radius; horizontaloffset <=
            radius; horizontaloffset++) {
            for (int verticaloffset = -radius; verticaloffset <= radius;
                verticaloffset++) {
                int currentx = xcoordinate + horizontaloffset;
                int currenty = ycoordinate + verticaloffset;

                if (currentx >= 0 && currentx < width && currenty >=
                    0 && currenty < height) {
                    Color color = pixelreader.getColor(currentx, currenty);
                    redsum += color.getRed();
                    greensum += color.getGreen();
                    bluesum += color.getBlue();
                    pixelsum++;
                }
            }
        }

        pixelwriter.setColor(xcoordinate, ycoordinate,
            new Color(redsum / pixelsum, greensum / pixelsum,
                bluesum / pixelsum, 1));
    }
}

```

16. Részlet a *FilterOperations.java*

Kép forgatása tetszőleges szögben

Ez a kódrészlet a kép tetszőleges szögben történő forgatását valósítja meg. A számítás során a kép középpontját veszem alapul, így a forgatás nem tolja el a képet. A `Math.cos` és `Math.sin` függvények segítségével kiszámolom, hogy az új pixel melyik eredeti pixelből származik. Ez egy klasszikus koordináta-transzformációs eljárás. Az *x* és *y* koordinátákat eltolom a középponthoz képest, majd visszahelyezem őket. Ez biztosítja a helyes forgatási eredményt. Ezután ellenőrzöm, hogy a számolt koordináták a kép határain belül vannak-e. Ha igen, akkor az eredeti pixel színét másolom át.

```

int sourcex = (int) (Math.cos(angle) * (xcoordinate - centerx) +
    Math.sin(angle) * (ycoordinate - centery) + centerx);
int sourcey = (int) (-Math.sin(angle) * (xcoordinate - centerx) +
    Math.cos(angle) * (ycoordinate - centery) + centery);

if (sourcex >= 0 && sourcex < width && sourcey >= 0 && sourcey < height) {
    pixelwriter.setColor(xcoordinate, ycoordinate,
        pixelreader.getColor(sourcex, sourcey));
    } else {
        pixelwriter.setColor(xcoordinate, ycoordinate, Color.BLACK);
    }
}

```

17. Részlet a *TransformOperations.java*

Bejelentkezés és felhasználónév átadása

Ebben a kódrészletben történik a felhasználó bejelentkeztetése az alkalmazásba. A rendszer először ellenőrzi, hogy a felhasználónév és a jelszó mezők ki vannak-e töltve. Amennyiben valamelyik mező üres, a program figyelmeztető üzenetet jelenít meg a felhasználó számára. Sikeres adatellenőrzés után az AuthService.login() metódus vizsgálja meg, hogy a megadott adatok helyesek-e. Ha a bejelentkezés sikeres, egy információs ablak jelenik meg „Sikeres belépés!” üzenettel. Ezután a program megnyitja az EditorView felületet, és paraméterként átadja a bejelentkezett felhasználó nevét a username változó segítségével. Az átadott felhasználónév később felhasználható például üdvözlő szöveg, profiladat vagy jogosultság megjelenítésére az alkalmazásban.

```
boolean success = AuthService.login(username, password);

if (success) {

Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle("");
    alert.setHeaderText(null);
    alert.setContentText("Sikeres belépés!");
    alert.showAndWait();

    new EditorView(primarystage, username);

}
```

18. Részlet a LoginView.java fájlból.

Felhasználónév megjelenítése az alkalmazásban

Ebben a kódrészletben történik a bejelentkezett felhasználó nevének megjelenítése a képszerkesztő alkalmazás felületén. Az EditorView osztály konstruktorában a program paraméterként kapja meg a felhasználó nevét a username változóban. A bal oldali panelen egy Label elem kerül létrehozásra, amely az „Üdv” szöveg mellett kiírja az aktuálisan bejelentkezett felhasználó nevét is. Ez személyesebb és felhasználóbarátabb megjelenést biztosít az alkalmazás számára.

```
Label welcomelabel = new Label("Üdv, " + username + "!");
welcomelabel.setStyle("-fx-text-fill:white;-fx-font-size:16px;-fx-font-weight:bold;");
```

19. Részlet a EditorView.java fájlból.

Visszavonás funkció megvalósítása képelőzmények segítségével

Ebben a kódrészletben a program a képszerkesztési műveletek visszavonását valósítja meg. Az alkalmazás egy Stack<Image> típusú veremet használ, amely eltárolja a korábbi képeket minden módosítás előtt. Minden szerkesztési művelet előtt a savehistory() metódus elmenti az aktuális képet az előzmények közé. A „Vissza” funkció az undoimage() metódus segítségével működik, amely kiveszi a legutóbbi képet a veremből és újra megjeleníti azt. Ez lehetővé teszi, hogy a felhasználó könnyen visszavonja az elrontott vagy nem kívánt módosításokat.

```
private final Stack<Image> imagehistory = new Stack<>();

private void savehistory() {
    if (currentimage != null) {
        imagehistory.push(currentimage);
    }
}

private void undoimage() {
    if (!imagehistory.isEmpty()) {
```

```

        currentimage = imagehistory.pop();
        imageview.setImage(currentimage);
    }
}

```

20. Részlet a *EditorView.java* fájlból.

Zöld színcsatorna alkalmazása a képre

Ez a metódus a kép zöld színcsatornájának alkalmazásáért felelős. Először ellenőrzi, hogy van-e jelenleg betöltött kép a `currentimage != null` feltétellel. Ha van kép, akkor a `savehistory()` metódus elmenti az aktuális állapotot a visszavonás funkcióhoz. Ezután meghívja a `ColorOperations.greenchannel(currentimage)` metódust, amely a tényleges képfeldolgozást végzi el egy másik osztályban. A feldolgozás során a kép csak a zöld színcsatornát tartja meg. Az eredményül kapott új kép visszakerül a `currentimage` változóba. Végül az `imageview.setImage(currentimage)` frissíti a felhasználói felületen megjelenített képet. A metódus tehát nem maga végzi a képfeldolgozást, hanem a `ColorOperations` osztály megfelelő függvényét hívja meg.

```

private void applygreenchannel() {
    if (currentimage != null) {
        savehistory();
        currentimage = ColorOperations.greenchannel(currentimage);
        imageview.setImage(currentimage);
    }
}

```

21. Részlet az *EditorView.java* fájlból.

Sliderek eseménykezelése és képmódosító műveletek

Ez a kódrészlet a különböző sliderek működését kezeli a képszerkesztő programban. Az első részben az `setOnMousePressed()` eseménykezelők figyelik, mikor kattint a felhasználó egy sliderre. Ilyenkor a `savehistory()` metódus elmenti az aktuális képállapotot, hogy később a visszavonás funkció használható legyen. Ezután a `valueProperty().addListener()` figyelők követik a slider értékének változását. Ha a felhasználó mozgatja a csúszkát, automatikusan meghívódik a megfelelő képfeldolgozó metódus. A `brightnessslider` a fényerőt, a `contrastslider` a kontrasztot módosítja. A `blurslider` homályosítást, a `mosaicslider` mozaik effektet alkalmaz. A `rotationslider` pedig a kép forgatását végzi el. A tényleges képfeldolgozás nem itt történik, hanem a megfelelő műveleti metódusokban.

```

brightnessslider.setOnMousePressed(event -> savehistory());
contrastslider.setOnMousePressed(event -> savehistory());
blurslider.setOnMousePressed(event -> savehistory());
mosaicslider.setOnMousePressed(event -> savehistory());
rotationslider.setOnMousePressed(event -> savehistory());

brightnessslider.valueProperty().addListener((o, ov, nv) ->
    applybrightness());
contrastslider.valueProperty().addListener((o, ov, nv) -> applycontrast());
blurslider.valueProperty().addListener((o, ov, nv) -> applyblur());
mosaicslider.valueProperty().addListener((o, ov, nv) -> applymosaic());
rotationslider.valueProperty().addListener((o, ov, nv) ->
    applyrotationangle());

```

22. Részlet az *EditorView.java* fájlból.

Felhasznált irodalom

- A féléves beadandóm elkészítéséhez mintaként az órai feladatokat használtam fel.
- A JavaFX fájlokban, főként az *EditorView.java*, *ColorOperations.java* és a *TransformOperations.java* fájlok elkészítésében, kódrészletekben a mesterséges intelligencia nyújtott segítséget.